Exercício 10.4: Desafio: checklist de produção e empacotamento

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1, Calico (NetworkPolicy funcional).
- Instalados para este exercício: **metrics-server** (Helm, `--kubelet-insecure-tls`, necessário no kind) para o HPA, e **kube-prometheus-stack 92.11.0** enxuto (sem Grafana, Alertmanager, node-exporter e kube-state-metrics) para o `ServiceMonitor` e a `PrometheusRule`.
- **Opção Kustomize.** `store-api` é a imagem endurecida do 10.2, agora em Debian 13 (o achado de lá), com `/metrics` (`prometheus_client`) e log JSON com `request_id`. `store-front` é o `nginx-unprivileged`. `store-worker` consome a fila do redis, como no 8.4.

Árvore

```
.
├── app
│   ├── Dockerfile
│   ├── main.py
│   └── requirements.txt
├── k8s
│   ├── base
│   │   ├── kustomization.yaml
│   │   ├── redis.yaml
│   │   ├── serviceaccounts.yaml
│   │   ├── store-api.yaml
│   │   ├── store-front.yaml
│   │   └── store-worker.yaml
│   └── overlays
│       ├── dev
│       │   ├── kustomization.yaml
│       │   └── namespace.yaml
│       └── prod
│           ├── hpa.yaml
│           ├── kustomization.yaml
│           ├── limitrange.yaml
│           ├── namespace.yaml
│           ├── networkpolicy.yaml
│           ├── patch-store-api.yaml
│           ├── patch-store-front.yaml
│           ├── pdb.yaml
│           ├── priorityclass.yaml
│           ├── prometheusrule.yaml
│           ├── resourcequota.yaml
│           └── servicemonitor.yaml
├── Makefile
└── README.md
```

Base e overlays

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
labels:
  - pairs:
      app.kubernetes.io/part-of: fiap-store
resources:
  - store-api.yaml
  - store-front.yaml
  - store-worker.yaml
  - redis.yaml
  - serviceaccounts.yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: store-api
  strategy:
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    metadata:
      labels:
        app: store-api
    spec:
      serviceAccountName: store-api
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        runAsGroup: 65532
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: store-api
          image: store-api
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
```

```
            containerPort: 8000
          env:
            - name: APP_ENV
              value: dev
          readinessProbe:
            httpGet:
              path: /ready
              port: http
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: http
            periodSeconds: 10
            failureThreshold: 3
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              memory: 128Mi
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: tmp
              mountPath: /tmp
      volumes:
        - name: tmp
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: store-api
  labels:
    app: store-api
spec:
  selector:
    app: store-api
  ports:
    - name: http
      port: 80
      targetPort: http
```

Os outros três Deployments da base seguem o mesmo padrão de `securityContext`: `store-front.yaml`, `store-worker.yaml` e `redis.yaml`, além de `serviceaccounts.yaml`.

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: store-dev
resources:
  - ../../base
  - namespace.yaml
images:
  - name: store-api
    newTag: "1.0"
# base ja e 1 replica, APP_ENV=dev e recursos pequenos: dev nao precisa de patch
```

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: store-prod
resources:
  - ../../base
  - namespace.yaml
  - limitrange.yaml
  - resourcequota.yaml
  - pdb.yaml
  - hpa.yaml
  - servicemonitor.yaml
  - networkpolicy.yaml
  - priorityclass.yaml
  - prometheusrule.yaml
images:
  - name: store-api
    newTag: "1.3" # imagem local carregada no kind: tag versionada e nunca reutilizada
  - name: nginxinc/nginx-unprivileged
    digest: sha256:f5116ed4606f3eb015dd4a8e28cb9b351476dc0ad4bc6e9f93b19cec130cdf03 # 1.30-alpine
  - name: redis
    digest: sha256:57e4255cc971b1c8d6f8a6ca283ac01bc3ecdd914273ef6bfaede87aaecd6541 # 7-alpine
# store-api fora da lista: quem manda nas replicas dela e o HPA (minReplicas: 3)
replicas:
  - name: store-front
    count: 3
  - name: store-worker
    count: 2
patches:
  - path: patch-store-api.yaml
  # sem spec.replicas: a contagem e do HPA (doc oficial: "Migrating Deployments ... to horizontal
autoscaling")
  - target:
      kind: Deployment
      name: store-api
    patch: |-
      - op: remove
        path: /spec/replicas
  - path: patch-store-front.yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
spec:
  template:
    spec:
      priorityClassName: store-critica
      terminationGracePeriodSeconds: 45
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: store-api
      containers:
        - name: store-api
          env:
            - name: APP_ENV
              value: prod
          # medido sob carga (16 mil req em 75 s): 65-123m de CPU e 38Mi por Pod
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              memory: 64Mi
          lifecycle:
            preStop:
              sleep: # acao nativa: o distroless nao tem /bin/sleep
                seconds: 10
```

```yaml
# default-deny de entrada e saida; tudo abaixo e liberacao explicita
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
# DNS para todos os Pods do namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: permite-dns
```

```yaml
spec:
  podSelector: {}
  policyTypes: ["Egress"]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
---
# front: recebe de qualquer origem, so fala com a api
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: store-front
spec:
  podSelector:
    matchLabels:
      app: store-front
  policyTypes: ["Ingress", "Egress"]
  ingress:
    - ports:
        - port: 8080
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: store-api
      ports:
        - port: 8000
---
# api: recebe do front e do Prometheus (namespace monitoring)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: store-api
spec:
  podSelector:
    matchLabels:
      app: store-api
  policyTypes: ["Ingress"]
```

```yaml
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: store-front
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: monitoring
      ports:
        - port: 8000
---
# worker: so fala com o redis
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: store-worker
spec:
  podSelector:
    matchLabels:
      app: store-worker
  policyTypes: ["Egress"]
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: redis
      ports:
        - port: 6379
---
# redis: so recebe do worker
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: redis
spec:
  podSelector:
    matchLabels:
      app: redis
  policyTypes: ["Ingress"]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: store-worker
      ports:
        - port: 6379
```

Os outros arquivos de prod são curtos e aparecem na saída completa do `kustomize` no fim deste documento: `namespace.yaml` (PSA `restricted`), `pdb.yaml`, `hpa.yaml`, `servicemonitor.yaml`, `prometheusrule.yaml`, `priorityclass.yaml`, `limitrange.yaml`, `resourcequota.yaml` e `patch-store-front.yaml`.

Makefile

```
# uso: make deploy-dev | deploy-prod | scan | diff | destroy
export DOCKER_CONTEXT ?= rootless
export DOCKER_HOST ?= unix:///run/user/1000/docker.sock
export KUBECONFIG ?= $(abspath ../../assets/kubeconfig)
IMAGENS = store-api:1.3 nginxinc/nginx-unprivileged:1.30-alpine redis:7-alpine

.PHONY: deploy-dev deploy-prod scan diff destroy

deploy-dev:
    kubectl apply -k k8s/overlays/dev
    kubectl -n store-dev rollout status deploy --timeout=180s

deploy-prod:
    kubectl apply -k k8s/overlays/prod
    kubectl -n store-prod rollout status deploy --timeout=180s

scan:
    for i in $(IMAGENS); do trivy image -q --exit-code 1 --ignore-unfixed --severity CRITICAL $$i ||
exit 1; done

diff:
    -kubectl diff -k k8s/overlays/dev
    -kubectl diff -k k8s/overlays/prod

destroy:
    kubectl delete -k k8s/overlays/prod --ignore-not-found
    kubectl delete -k k8s/overlays/dev --ignore-not-found
```

`make deploy-prod` é **um único comando** que sobe o ambiente inteiro (26 objetos: 4 Deployments, 3 Services, 4 ServiceAccounts, 6 NetworkPolicies, 2 PDBs, HPA, ServiceMonitor, PrometheusRule, PriorityClass, LimitRange, ResourceQuota e o Namespace).

Os dois ambientes no mesmo cluster

```
$ kubectl get all,pdb,hpa,netpol,quota -n store-prod
NAME                                READY   STATUS    RESTARTS   AGE
pod/redis-85d7b8d48b-mzbwt          1/1     Running   0          7m25s
pod/store-api-54868cdcc9-7bv9r      1/1     Running   0          2m48s
pod/store-api-54868cdcc9-97qgm      1/1     Running   0          2m48s
pod/store-api-54868cdcc9-bc287      1/1     Running   0          3m3s
pod/store-api-54868cdcc9-dt8fk      1/1     Running   0          4m56s
pod/store-api-54868cdcc9-hvnt5      1/1     Running   0          3m3s
pod/store-front-79cf9c7cb6-4wlnk    1/1     Running   0          3m42s
pod/store-front-79cf9c7cb6-sjmhk    1/1     Running   0          3m51s
pod/store-front-79cf9c7cb6-vc28k    1/1     Running   0          3m41s
pod/store-worker-85bc87457d-6cvn4   1/1     Running   0          7m24s
pod/store-worker-85bc87457d-tnz96   1/1     Running   0          7m25s

NAME                  TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
service/redis         ClusterIP   10.96.62.161   <none>        6379/TCP   10m
service/store-api     ClusterIP   10.96.106.83   <none>        80/TCP     10m
service/store-front   ClusterIP   10.96.30.93    <none>        80/TCP     10m

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/redis         1/1     1            1           10m
deployment.apps/store-api     5/5     5            5           10m
deployment.apps/store-front   3/3     3            3           10m
deployment.apps/store-worker  2/2     2            2           10m

NAME                                      DESIRED   CURRENT   READY   AGE

NAME                                          REFERENCE              TARGETS       MINPODS
MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/store-api   Deployment/store-api   cpu: 2%/70%   3         10
5         10m

NAME                                    MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
poddisruptionbudget.policy/store-api    2               N/A               3                     10m
poddisruptionbudget.policy/store-front  2               N/A               1                     10m

NAME                                       POD-SELECTOR       AGE
networkpolicy.networking.k8s.io/default-deny   <none>             10m
networkpolicy.networking.k8s.io/permite-dns    <none>             10m
networkpolicy.networking.k8s.io/redis          app=redis          10m
networkpolicy.networking.k8s.io/store-api      app=store-api      10m
networkpolicy.networking.k8s.io/store-front    app=store-front    10m
networkpolicy.networking.k8s.io/store-worker   app=store-worker   10m

NAME                     REQUEST                                                 LIMIT
```

```
AGE
resourcequota/store-prod   pods: 11/30, requests.cpu: 600m/3, requests.memory: 480Mi/3Gi
limits.memory: 544Mi/4Gi   10m

$ kubectl get all,pdb,hpa,netpol,quota -n store-dev
NAME                                READY   STATUS    RESTARTS   AGE
pod/redis-64f9ccb86b-4srjx          1/1     Running   0          7m28s
pod/store-api-5d97797fbd-kr7jp      1/1     Running   0          7m28s
pod/store-front-79794bb77c-7l4gb    1/1     Running   0          3m59s
pod/store-worker-7988bf6988-pxl9w   1/1     Running   0          7m28s

NAME                  TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
service/redis         ClusterIP   10.96.48.0     <none>        6379/TCP   10m
service/store-api     ClusterIP   10.96.130.71   <none>        80/TCP     10m
service/store-front   ClusterIP   10.96.52.76    <none>        80/TCP     10m

NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/redis          1/1     1            1           10m
deployment.apps/store-api      1/1     1            1           10m
deployment.apps/store-front    1/1     1            1           10m
deployment.apps/store-worker   1/1     1            1           10m

NAME                                      DESIRED   CURRENT   READY   AGE
```

```
store-dev:  redis=1 redis:7-alpine
            store-api=1 store-api:1.0
            store-front=1 nginxinc/nginx-unprivileged:1.30-alpine
            store-worker=1 redis:7-alpine
store-prod: redis=1 redis@sha256:57e4255cc971b1c8d6f8a6ca283ac01bc3ecdd914273ef6bfaede87aaecd6541
            store-api=(HPA 3..10) store-api:1.3
            store-front=3 nginxinc/nginx-
unprivileged@sha256:f5116ed4606f3eb015dd4a8e28cb9b351476dc0ad4bc6e9f93b19cec130cdf03
            store-worker=2
redis@sha256:57e4255cc971b1c8d6f8a6ca283ac01bc3ecdd914273ef6bfaede87aaecd6541
```

Réplicas, tags e digests diferentes. `APP_ENV` também muda (`dev` x `prod`), visto pelo `/info`:

```
$ kubectl -n store-dev exec deploy/store-front -- wget -qO- http://store-api/info
{"versao":"1.0","env":"dev","db_host":"nao-configurado","pod":"store-api-86d7f944cd-29wck"}
$ kubectl -n store-prod exec deploy/store-front -- wget -qO- http://store-api/info
{"versao":"1.3","env":"prod","db_host":"nao-configurado","pod":"store-api-54868cdcc9-dt8fk"}
```

Isolamento comprovado:

```
# front de dev tentando a api de prod: bloqueado pelas NetworkPolicies de store-prod
$ kubectl -n store-dev exec deploy/store-front -- wget -qO- -T 3 http://store-api.store-prod/info
wget: download timed out

# worker de prod tentando a api de prod: o egress dele só permite o redis
$ kubectl -n store-prod exec deploy/store-worker -- sh -c 'timeout 4 redis-cli -h store-api -p 80
PING'
Terminated  (exit=143)

# o caminho permitido funciona: a fila é consumida pelos 2 workers
$ kubectl -n store-prod exec deploy/redis -- redis-cli RPUSH pedidos pedido-1 pedido-2 pedido-3
3
[pod/store-worker-...-8v57g/store-worker] pedido-2
[pod/store-worker-...-8v57g/store-worker] pedido-3
[pod/store-worker-...-crrsv/store-worker] pedido-1
```

Nenhum recurso de `store-prod` viola o PSA `restricted`. Reavaliar o namespace força o API server a checar todos os Pods existentes. Como contraprova de que a checagem funciona, o mesmo comando em `kube-system` lista violações:

```
$ kubectl label --dry-run=server --overwrite ns store-prod pod-security.kubernetes.io/enforce-
version=latest
namespace/store-prod labeled (server dry run)                ← nenhum aviso

$ kubectl label --dry-run=server --overwrite ns kube-system pod-
security.kubernetes.io/enforce=restricted
Warning: existing pods in namespace "kube-system" violate the new PodSecurity enforce level
"restricted:latest"
Warning: calico-node-25wws (and 2 other pods): host namespaces, privileged, allowPrivilegeEscalation
≠ false, ...
```

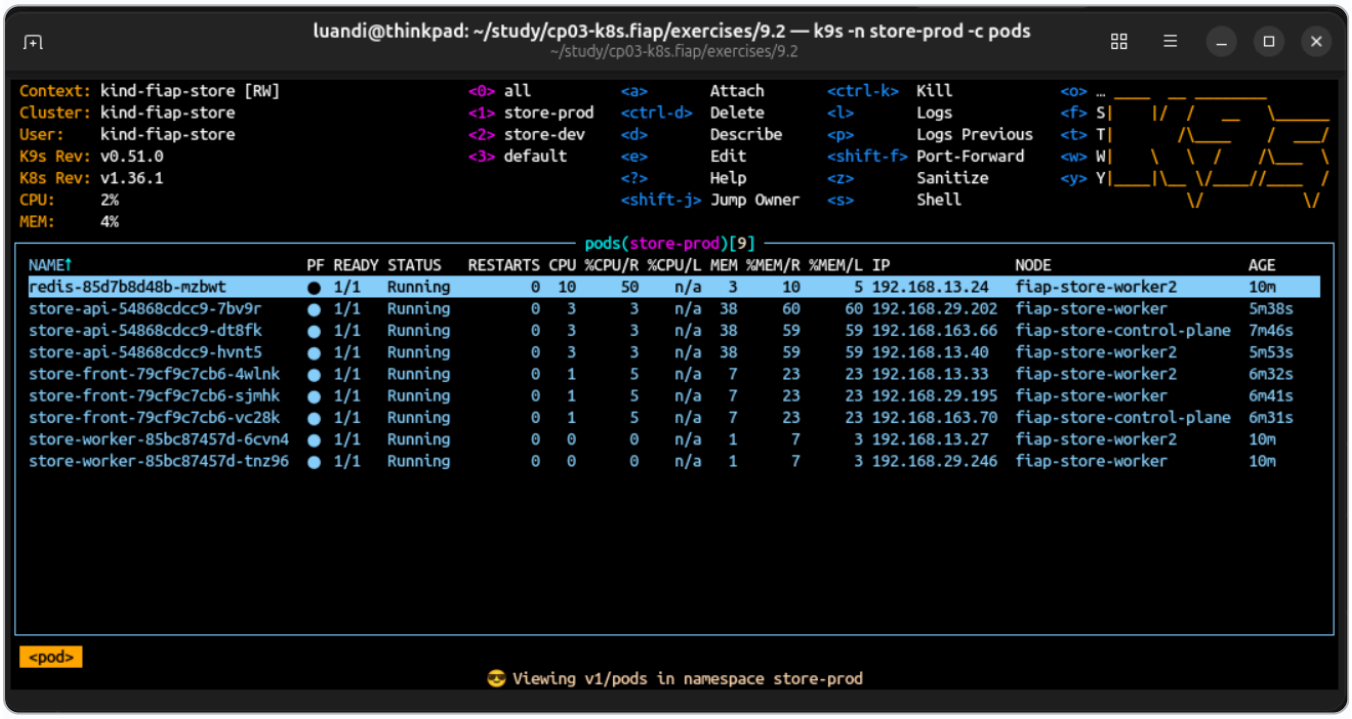
Observabilidade e autoescala funcionando:

```
# Prometheus: os alvos do ServiceMonitor
store-api http://192.168.13.14:8000/metrics/ up
store-api http://192.168.163.123:8000/metrics/ up
store-api http://192.168.29.237:8000/metrics/ up

# PrometheusRule carregada
StoreApiPoucasReplicas state=inactive health=ok

# HPA sob carga (16.039 requisições em 75 s, disparadas do front)
Normal  SuccessfulRescale  horizontal-pod-autoscaler  New size: 6; reason: cpu resource utilization
(percentage of request) above target

# log JSON da api
{"request_id": "5c71d395c7b34b11b817dc4b309d0944", "metodo": "GET", "rota": "/produtos", "status":
200, "ms": 1.0}
```



Achados durante a montagem

1. `make scan` **reprovou o front.** O `nginx-unprivileged:1.27-alpine` (Alpine 3.21.3) tinha a CVE-2026-31789 CRITICAL corrigível em `libssl3` / `libcrypto3`. A troca para `1.30-alpine` passou. É a porta de qualidade fazendo o trabalho dela.
2. **HPA x** `spec.replicas`**.** Com `replicas: 3` no overlay, cada `make deploy-prod` derrubava para 3 o que o HPA tinha escalado para 6. Tirar a api da lista do Kustomize foi pior: o `replicas: 1` da base foi aplicado e **o prod caiu para 1 réplica** até o HPA reagir (`New size: 3; reason: Current number of replicas below Spec.MinReplicas`). A correção segue a doc oficial (”Migrating

Deployments and StatefulSets to horizontal autoscaling"): um patch JSON `remove /spec/replicas` no overlay de prod, mais `kubectl apply set-last-applied` antes do próximo apply, para o client-side apply não interpretar a remoção do campo como "volte ao padrão 1". Depois disso, applies seguidos mantiveram a contagem do HPA (`6 → 6 → 6`).

3. **A `PrometheusRule` não carregava.** O Prometheus do chart só lê regras com `release: monitoring` (`ruleSelector`). Com o label, ela aparece com `health=ok`.
4. `preStop` **em distroless.** Não existe `/bin/sleep` na imagem, então o `exec: ["sleep", "10"]` do 10.3 falharia. Foi usada a ação nativa `preStop.sleep.seconds`.
5. `/metrics` **responde em** `/metrics/`**.** O `mount` do `prometheus_client` redireciona `/metrics` para `/metrics/` com 307. O `ServiceMonitor` aponta direto para `/metrics/`.

CHECKLIST de produção (seção 10.14) aplicado ao `store-prod`

Legenda: `[x]` atendido · `[ ]` não atendido · `[-]` não aplicável.

Placar: 30 `[x]` + 3 `[-]` = 33 de 41 (80,5%). 8 `[ ]`, todos com justificativa.

Imagem e build

- [x] **Imagem base mínima, multi-stage.** `store-api:1.3` é multi-stage com base distroless `python3-debian13`. Front e redis usam imagens Alpine oficiais.
- [x] `USER` **numérico não-root.** `store-api`: `USER 65532:65532`. `nginx-unprivileged`: `USER 101`. A imagem do redis não define usuário, então o Pod força `runAsUser: 999`.
- [x] **Tag imutável ou digest.** Em prod, nginx e redis são fixados por `digest` no overlay. A `store-api` é imagem local carregada no kind (sem registry, não há digest de registry), com tag versionada que nunca é reaproveitada (`1.3`).
- [x] **Scan sem CRITICAL corrigível.** O `make scan` roda `trivy --exit-code 1 --ignore-unfixed --severity CRITICAL`. Ele **reprovou** o nginx 1.27 (CVE-2026-31789 no OpenSSL), o que levou à troca para o 1.30.
- [ ] **SBOM e Cosign.** O SBOM CycloneDX foi gerado no 10.2, mas não há assinatura Cosign: sem registry, não há onde publicar a assinatura.
- [x] `.dockerignore`**.** Cobre `.git`, `.env`, `*.pem` e `*.key`.
- [-] **Registry privado com scan on push.** No lab, as imagens são carregadas direto nos nós com `kind load`. Em AWS seria o ECR com scan on push e lifecycle policy.

Segurança do workload

- [x] `runAsNonRoot`, `runAsUser`, `allowPrivilegeEscalation: false` nos 4 Deployments (base).
- [x] `readOnlyRootFilesystem` **com** `emptyDir`. `/tmp` na api e no front, `/data` no redis.
- [x] `drop: ["ALL"]` **e** `seccompProfile: RuntimeDefault` nos 4.
- [x] **PSA.** `store-prod` em `enforce: restricted`. `store-dev` em `baseline` com `warn: restricted`, para dev avisar sem bloquear experimentos.
- [x] `automountServiceAccountToken: false` no Pod e na ServiceAccount.
- [x] **ServiceAccount dedicada, menor privilégio.** Uma por app, sem nenhum RoleBinding, porque nenhum app usa a API.
- [x] **NetworkPolicy default-deny com allow explícito.** 6 políticas, testadas: o front de prod chega na api, o worker não, e o front de dev não chega na api de prod.
- [ ] **Secrets fora do Git, encryption at rest.** O pacote não tem Secrets, mas o etcd do kind não tem `EncryptionConfiguration`. Fica para o Sealed Secrets do capítulo 11.
- [x] **Sem** `hostNetwork`, `hostPID`, `hostPath` **ou** `privileged`. Comprovado pela reavaliação do PSA `restricted`, sem nenhum aviso.

Confiabilidade

- [x] `replicas ≥ 2` **e spread.** api com 3 a 10 réplicas (HPA) e `topologySpreadConstraints` por nó. Front com 3 e worker com 2. O kind tem 3 nós, então o spread é por `hostname`; em EKS seria `topology.kubernetes.io/zone`.
- [x] `readinessProbe` **e** `livenessProbe` **distintos.** Na api, `/ready` e `/health`. O front tem só readiness, porque o nginx não trava sem cair.
- [-] `startupProbe`**.** O boot leva menos de 2 s.
- [x] `preStop` **e grace.** `preStop.sleep` nativo de 10 s (o distroless não tem `/bin/sleep`) e grace de 45 s na api e 30 s no front.
- [x] **Trata SIGTERM, exec form.** O uvicorn é o PID 1. O distroless nem tem shell, então shell form é impossível.
- [x] **PDB compatível.** `minAvailable: 2` com 3 ou mais réplicas, o que deixa 1 disrupção permitida.
- [x] `maxUnavailable: 0` (base).
- [x] **HPA com métrica de carga real.** CPU a 70%. Sob carga, escalou de 3 para 6. O worker escalaria por fila (KEDA, exercício 8.4).

Recursos

- [x] `requests` **por medição.** Carga de 16.039 requisições em 75 s: a api ficou em 65 a 123m de CPU e 38Mi, e o front em 6 a 11Mi. Daí os requests de 100m/64Mi e 20m/32Mi.
- [x] `limits` **de memória = request.** api 64Mi/64Mi e front 32Mi/32Mi. A QoS fica `Burstable` porque não há limite de CPU (item seguinte).
- [x] `limits` **de CPU omitidos e justificados.** Limite de CPU causa throttling mesmo com o nó ocioso. Sem limite não existe cota CFS, então `container_cpu_cfs_throttled_periods_total` nem é emitido para esses containers (consulta vazia no Prometheus). O que se monitora é o uso real em `container_cpu_usage_seconds_total` (36 séries de `store-prod`, raspadas do kubelet) contra o request.
- [x] `LimitRange` **e** `ResourceQuota` em `store-prod`.
- [x] `priorityClassName`. `store-critica` (100000) na api.

Observabilidade

- [x] **Logs JSON em stdout com** `request_id`**.** Middleware na api, que reaproveita o `x-request-id` do cliente e devolve o mesmo id no header.
- [ ] **Agregação fora do cluster.** Não há Loki nem CloudWatch no lab.
- [x] `/metrics` **e** `ServiceMonitor`. 3 alvos `up=1` no Prometheus.
- [ ] **Dashboard dos 4 sinais dourados.** O Grafana foi desligado no chart para poupar recursos no kind.
- [x] **Alerta acionável e SLO.** PrometheusRule `StoreApiPoucasReplicas` (`health=ok`). O SLO está no comentário da regra e o runbook vem abaixo.
- [ ] **Tracing (OpenTelemetry).** Não instrumentado.

Operação

- [ ] **GitOps.** Os manifestos estão em Git com Kustomize, mas são aplicados por `make`. O GitOps (Argo CD) entra no capítulo 11.
- [x] **Runbook.** Ver abaixo.
- [-] **Backup e restauração.** Não há estado persistente: o redis é uma fila efêmera (`--save ""`), e em produção seria SQS (recomendação do 8.4).
- [x] **Plano de upgrade.** Ver abaixo.
- [ ] **Staging equivalente a produção.** O `store-dev` é diferente por desenho, como pede o enunciado (1 réplica, `baseline`, sem PDB).

- [ ] **Custos por namespace.** Os labels `app.kubernetes.io/part-of` existem, mas não há OpenCost/Kubecost instalado.

Runbook: StoreApiPoucasReplicas

1. `kubectl -n store-prod get pods -l app=store-api -o wide`: identificar os Pods que não estão `1/1`.
2. `kubectl -n store-prod describe pod <pod>`: procurar em Events `FailedScheduling` (quota ou recursos), `Unhealthy` (probe) ou `OOMKilled`.
3. `kubectl -n store-prod logs <pod> --previous`: erro de aplicação. Use o `request_id` do log JSON para correlacionar.
4. Se o problema começou num deploy: `kubectl -n store-prod rollout undo deploy/store-api`.
5. Se for `FailedScheduling` por quota: `kubectl -n store-prod describe quota`. Não aumente a quota sem revisar o HPA (`maxReplicas`).

Plano de upgrade

- **Cluster:** uma minor por vez (1.36 → 1.37), control plane antes dos nós. Antes de subir, rodar `kubectl apply -k ... --dry-run=server` contra a versão nova para pegar APIs removidas. Os PDBs garantem a drenagem nó a nó sem queda.
- **Imagens:** bump de tag ou digest no overlay, `make scan`, `make diff`, `make deploy-prod`. O rollback é o `rollout undo` ou o digest anterior no Git.
- **Operadores (kube-prometheus-stack, KEDA, metrics-server):** versões fixadas no Helm. Sobem primeiro em dev e ficam uma semana lá antes de prod.

Componente serverless

Deixa de fazer sentido no Lambda: "`preStop` (sleep de 5 a 15 s) + `terminationGracePeriodSeconds`" (junto com PDB e `maxUnavailable: 0`). Esses itens existem porque, no Kubernetes, **nós** controlamos o ciclo de vida do processo: decidimos quando um Pod sai, e por isso temos de esconder a corrida entre a remoção dos Endpoints e o SIGTERM (exercício 10.3). No Lambda não existe Service, nem Endpoints, nem processo de longa duração que alguém derruba no meio de uma requisição. A AWS decide quando um ambiente de execução nasce e morre, e só o remove do roteamento quando ele está ocioso. A responsabilidade **se move** para o

provedor porque o componente (orquestração e roteamento de instâncias) deixou de ser nosso.

Continua exatamente igual: "Scan sem CVE CRITICAL corrigível". O código da `store-api` e o `requirements.txt` (`fastapi`, `starlette`, `prometheus-client`) vão para dentro do pacote do Lambda do mesmo jeito. Se a `starlette` tem a CVE-2026-54283, a função continua vulnerável. Pelo modelo de responsabilidade compartilhada, a AWS corrige o SO e o runtime gerenciado, mas **o código e as dependências do cliente são sempre do cliente**. A linha não se move porque o artefato que carrega o risco continua sendo produzido por nós.

(O Knative Service opcional não foi feito.)

Saída de `kubectl kustomize k8s/overlays/prod`

```yaml
apiVersion: v1
kind: Namespace
metadata:
  labels:
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: v1.36
    pod-security.kubernetes.io/warn: restricted
  name: store-prod
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: store-prod
  namespace: store-prod
spec:
  hard:
    limits.memory: 4Gi
    pods: "30"
    requests.cpu: "3"
    requests.memory: 3Gi
---
apiVersion: v1
automountServiceAccountToken: false
kind: ServiceAccount
metadata:
  labels:
    app.kubernetes.io/part-of: fiap-store
  name: redis
  namespace: store-prod
```

```yaml
---
apiVersion: v1
automountServiceAccountToken: false
kind: ServiceAccount
metadata:
  labels:
    app.kubernetes.io/part-of: fiap-store
  name: store-api
  namespace: store-prod
---
apiVersion: v1
automountServiceAccountToken: false
kind: ServiceAccount
metadata:
  labels:
    app.kubernetes.io/part-of: fiap-store
  name: store-front
  namespace: store-prod
---
apiVersion: v1
automountServiceAccountToken: false
kind: ServiceAccount
metadata:
  labels:
    app.kubernetes.io/part-of: fiap-store
  name: store-worker
  namespace: store-prod
---
apiVersion: v1
kind: Service
metadata:
  labels:
    app: redis
    app.kubernetes.io/part-of: fiap-store
  name: redis
  namespace: store-prod
spec:
  ports:
  - name: redis
    port: 6379
    targetPort: redis
  selector:
    app: redis
---
apiVersion: v1
kind: Service
metadata:
  labels:
```

```
    app: store-api
    app.kubernetes.io/part-of: fiap-store
  name: store-api
  namespace: store-prod
spec:
  ports:
  - name: http
    port: 80
    targetPort: http
  selector:
    app: store-api
---
apiVersion: v1
kind: Service
metadata:
  labels:
    app: store-front
    app.kubernetes.io/part-of: fiap-store
  name: store-front
  namespace: store-prod
spec:
  ports:
  - name: http
    port: 80
    targetPort: http
  selector:
    app: store-front
---
apiVersion: v1
kind: LimitRange
metadata:
  name: padrao
  namespace: store-prod
spec:
  limits:
  - default:
      memory: 128Mi
    defaultRequest:
      cpu: 50m
      memory: 64Mi
    max:
      memory: 512Mi
    type: Container
---
apiVersion: scheduling.k8s.io/v1
description: store-api em producao
kind: PriorityClass
metadata:
```

```
  name: store-critica
value: 100000
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app.kubernetes.io/part-of: fiap-store
  name: redis
  namespace: store-prod
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      automountServiceAccountToken: false
      containers:
      - args:
        - --save
        - ""
        - --appendonly
        - "no"
        image: redis@sha256:57e4255cc971b1c8d6f8a6ca283ac01bc3ecdd914273ef6bfaede87aaecd6541
        name: redis
        ports:
        - containerPort: 6379
          name: redis
        readinessProbe:
          tcpSocket:
            port: redis
        resources:
          limits:
            memory: 64Mi
          requests:
            cpu: 20m
            memory: 32Mi
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop:
            - ALL
          readOnlyRootFilesystem: true
        volumeMounts:
```

```
        - mountPath: /data
          name: data
      securityContext:
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 999
        seccompProfile:
          type: RuntimeDefault
      serviceAccountName: redis
      volumes:
      - emptyDir: {}
        name: data
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app.kubernetes.io/part-of: fiap-store
  name: store-api
  namespace: store-prod
spec:
  selector:
    matchLabels:
      app: store-api
  strategy:
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: store-api
    spec:
      automountServiceAccountToken: false
      containers:
      - env:
        - name: APP_ENV
          value: prod
        image: store-api:1.3
        imagePullPolicy: IfNotPresent
        lifecycle:
          preStop:
            sleep:
              seconds: 10
        livenessProbe:
          failureThreshold: 3
          httpGet:
            path: /health
```

```
            port: http
          periodSeconds: 10
        name: store-api
        ports:
        - containerPort: 8000
          name: http
        readinessProbe:
          httpGet:
            path: /ready
            port: http
          periodSeconds: 5
        resources:
          limits:
            memory: 64Mi
          requests:
            cpu: 100m
            memory: 64Mi
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop:
            - ALL
          readOnlyRootFilesystem: true
        volumeMounts:
        - mountPath: /tmp
          name: tmp
      priorityClassName: store-critica
      securityContext:
        runAsGroup: 65532
        runAsNonRoot: true
        runAsUser: 65532
        seccompProfile:
          type: RuntimeDefault
      serviceAccountName: store-api
      terminationGracePeriodSeconds: 45
      topologySpreadConstraints:
      - labelSelector:
          matchLabels:
            app: store-api
        maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: ScheduleAnyway
      volumes:
      - emptyDir: {}
        name: tmp
---
apiVersion: apps/v1
kind: Deployment
```

```yaml
metadata:
  labels:
    app.kubernetes.io/part-of: fiap-store
  name: store-front
  namespace: store-prod
spec:
  replicas: 3
  selector:
    matchLabels:
      app: store-front
  strategy:
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: store-front
    spec:
      automountServiceAccountToken: false
      containers:
      - image: nginxinc/nginx-
unprivileged@sha256:f5116ed4606f3eb015dd4a8e28cb9b351476dc0ad4bc6e9f93b19cec130cdf03
        lifecycle:
          preStop:
            sleep:
              seconds: 10
        name: store-front
        ports:
        - containerPort: 8080
          name: http
        readinessProbe:
          httpGet:
            path: /
            port: http
        resources:
          limits:
            memory: 32Mi
          requests:
            cpu: 20m
            memory: 32Mi
        securityContext:
          allowPrivilegeEscalation: false
          capabilities:
            drop:
            - ALL
          readOnlyRootFilesystem: true
        volumeMounts:
```

```
        - mountPath: /tmp
          name: tmp
      securityContext:
        runAsGroup: 101
        runAsNonRoot: true
        runAsUser: 101
        seccompProfile:
          type: RuntimeDefault
      serviceAccountName: store-front
      terminationGracePeriodSeconds: 30
      volumes:
      - emptyDir: {}
        name: tmp
---
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app.kubernetes.io/part-of: fiap-store
  name: store-worker
  namespace: store-prod
spec:
  replicas: 2
  selector:
    matchLabels:
      app: store-worker
  template:
    metadata:
      labels:
        app: store-worker
    spec:
      automountServiceAccountToken: false
      containers:
      - args:
        - while true; do redis-cli -h redis BLPOP pedidos 0; done
        command:
        - sh
        - -c
        image: redis@sha256:57e4255cc971b1c8d6f8a6ca283ac01bc3ecdd914273ef6bfaede87aaecd6541
        name: store-worker
        resources:
          limits:
            memory: 32Mi
          requests:
            cpu: 10m
            memory: 16Mi
        securityContext:
          allowPrivilegeEscalation: false
```

```yaml
          capabilities:
            drop:
            - ALL
          readOnlyRootFilesystem: true
      securityContext:
        runAsGroup: 1000
        runAsNonRoot: true
        runAsUser: 999
        seccompProfile:
          type: RuntimeDefault
      serviceAccountName: store-worker
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: store-api
  namespace: store-prod
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: store-api
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: store-front
  namespace: store-prod
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: store-front
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: store-api
  namespace: store-prod
spec:
  maxReplicas: 10
  metrics:
  - resource:
      name: cpu
      target:
        averageUtilization: 70
        type: Utilization
    type: Resource
```

```yaml
  minReplicas: 3
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: store-api
---
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  labels:
    release: monitoring
  name: store-api
  namespace: store-prod
spec:
  groups:
  - name: store-api
    rules:
    - alert: StoreApiPoucasReplicas
      annotations:
        summary: store-api com menos de 2 replicas respondendo em store-prod
      expr: sum(up{namespace="store-prod", job="store-api"}) < 2
      for: 5m
      labels:
        severity: critical
---
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: store-api
  namespace: store-prod
spec:
  endpoints:
  - interval: 30s
    path: /metrics/
    port: http
  selector:
    matchLabels:
      app: store-api
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: store-prod
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

```
  - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: permite-dns
  namespace: store-prod
spec:
  egress:
  - ports:
    - port: 53
      protocol: UDP
    - port: 53
      protocol: TCP
    to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
      podSelector:
        matchLabels:
          k8s-app: kube-dns
  podSelector: {}
  policyTypes:
  - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: redis
  namespace: store-prod
spec:
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: store-worker
    ports:
    - port: 6379
  podSelector:
    matchLabels:
      app: redis
  policyTypes:
  - Ingress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: store-api
```

```
  namespace: store-prod
spec:
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: store-front
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: monitoring
    ports:
    - port: 8000
  podSelector:
    matchLabels:
      app: store-api
  policyTypes:
  - Ingress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: store-front
  namespace: store-prod
spec:
  egress:
  - ports:
    - port: 8000
    to:
    - podSelector:
        matchLabels:
          app: store-api
  ingress:
  - ports:
    - port: 8080
  podSelector:
    matchLabels:
      app: store-front
  policyTypes:
  - Ingress
  - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: store-worker
  namespace: store-prod
spec:
  egress:
```

```
  - ports:
    - port: 6379
    to:
    - podSelector:
        matchLabels:
          app: redis
  podSelector:
    matchLabels:
      app: store-worker
  policyTypes:
  - Egress
```

Critérios de aceite

Critério	Resultado
Um único comando sobe o ambiente	ok (`make deploy-prod`, ou `kubectl apply -k k8s/overlays/prod`)
Dois namespaces com configurações comprovadamente distintas	ok (réplicas, tag/digest, `APP_ENV`, PSA, PDB/HPA/NetworkPolicy só em prod)
Nada em `store-prod` viola o PSA `restricted`	ok (reavaliação sem avisos)
Pelo menos 80% do checklist em `[x]` / `[-]`	ok (33/41 = 80,5%)
Análise serverless com uma responsabilidade transferida e uma retida	ok